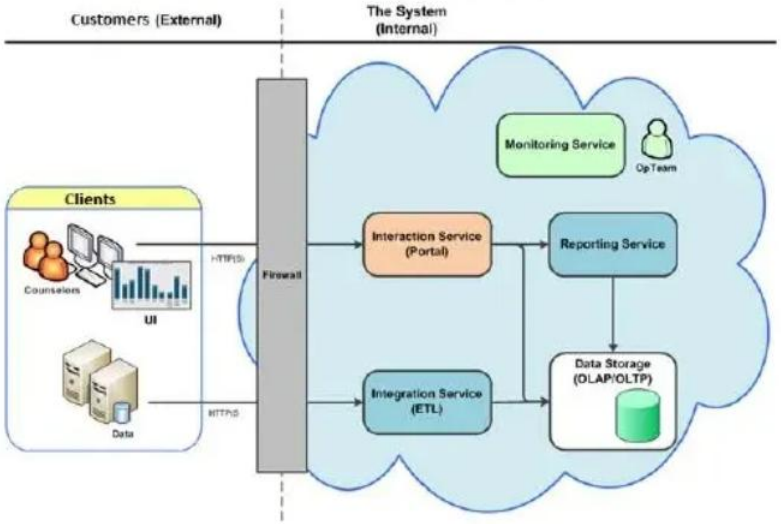
Chương 9
Thiết kế kiến trúc ứng dụng

Kiến trúc phần mềm

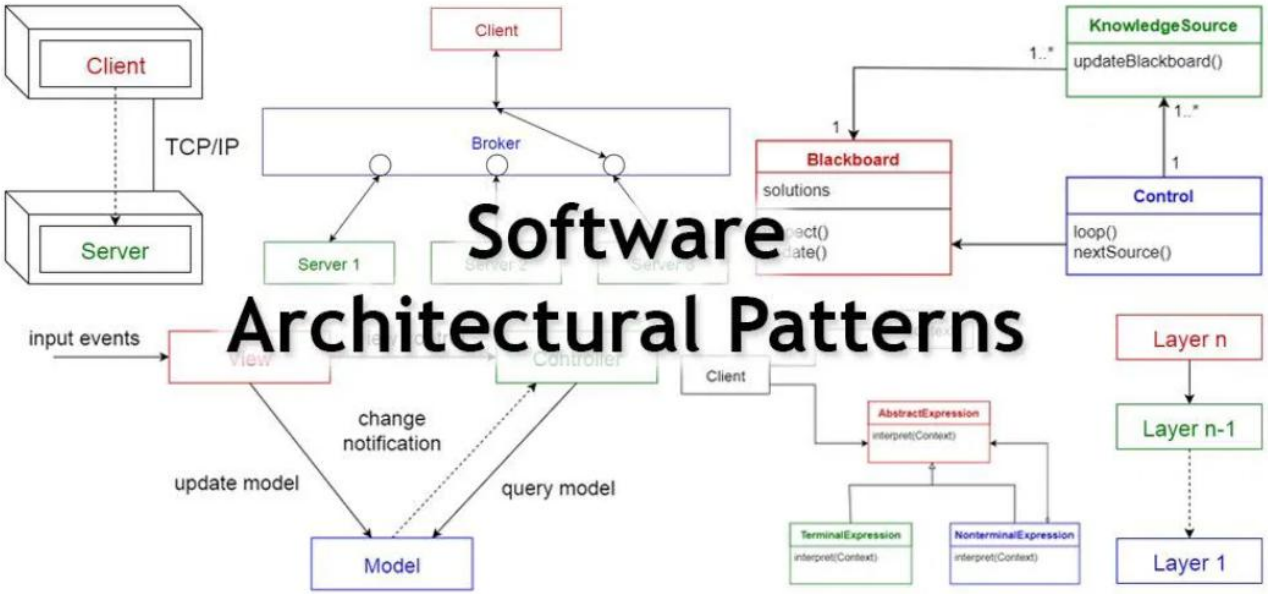
- Kiến trúc phần mềm là tập hợp các cấu trúc cần thiết để lập trình nên hệ thống phần mềm, bao gồm các phần tử, mối quan hệ giữa chúng và các thuộc tính của chúng.
- Kiến trúc phần mềm tác động đến hiệu suất, chất lượng và khả năng bảo trì của phần mềm.

The high-level breakdown of a system into its parts



Mẫu kiến trúc phần mềm

- Mẫu kiến trúc phần mềm (Software Architecture Patterns) là một giải pháp chung, có thể tái sử dụng để giải quyết một vấn đề thường gặp trong kiến trúc phần mềm. Nó giống như một khuôn mẫu có thể được áp dụng để tạo ra kiến trúc của một hệ thống phần mềm.

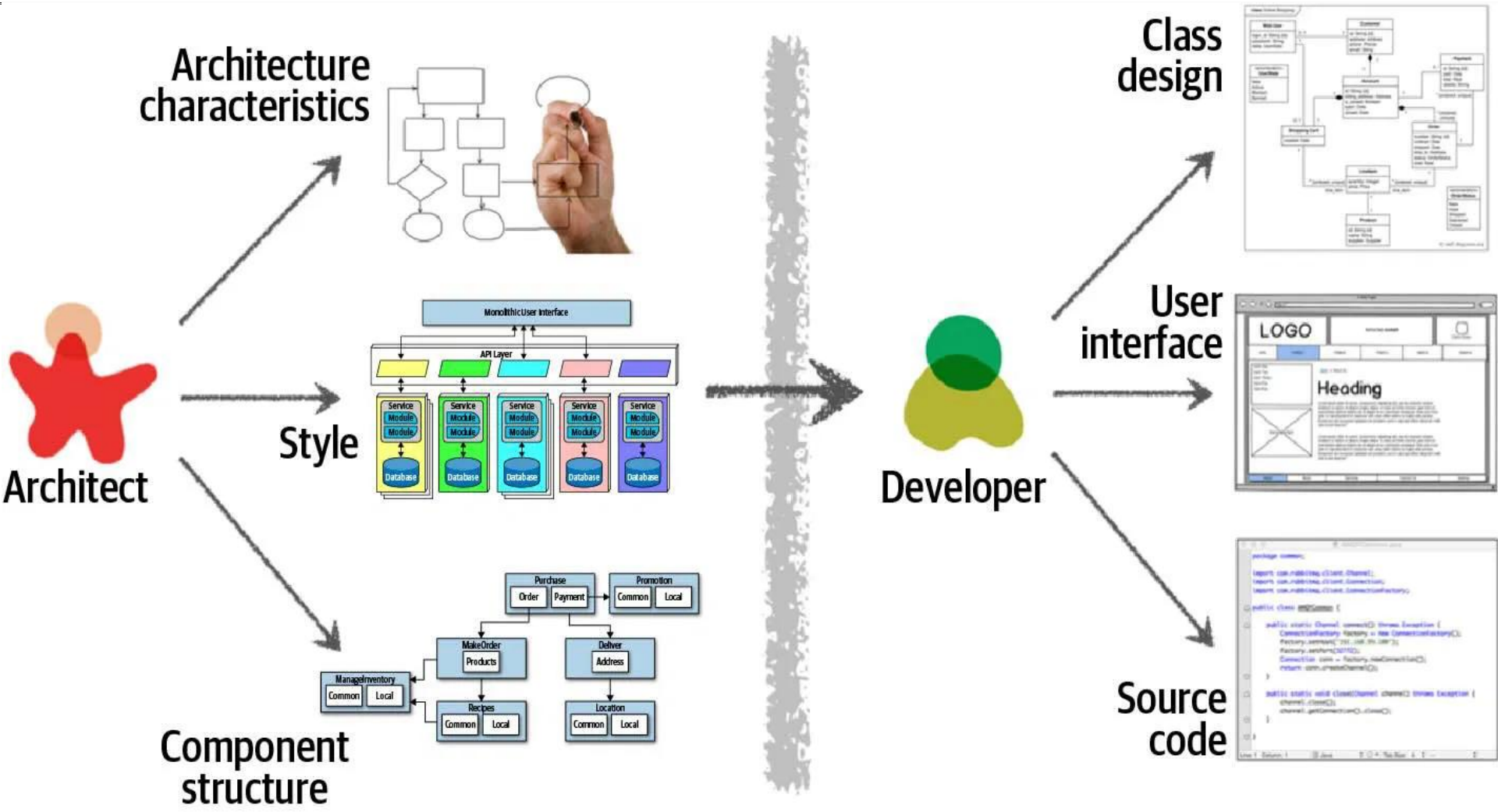


Vai trò của kiến trúc phần mềm

- Cung cấp khung làm việc rõ ràng:
  - Bao gồm cách các thành phần kỹ thuật tương tác với nhau. Nó như bản đồ giúp các nhà phát triển hiểu rõ cách xây dựng các thành phần của phần mềm.
- Đảm bảo hiệu suất và khả năng mở rộng:
  - Kiến trúc tốt cho phép hệ thống xử lý tải lớn và mở rộng dễ dàng khi cần thiết.
- Tối ưu hóa ngân sách và tài nguyên:
  - Kiến trúc hợp lý giúp tránh lãng phí nguồn lực và thời gian phát triển, do có kế hoạch và cấu trúc rõ ràng từ đầu.
- Dễ dàng bảo trì và cập nhật:
  - Cho phép sửa đổi hoặc thêm tính năng mới mà không làm ảnh hưởng đến toàn bộ hệ thống.

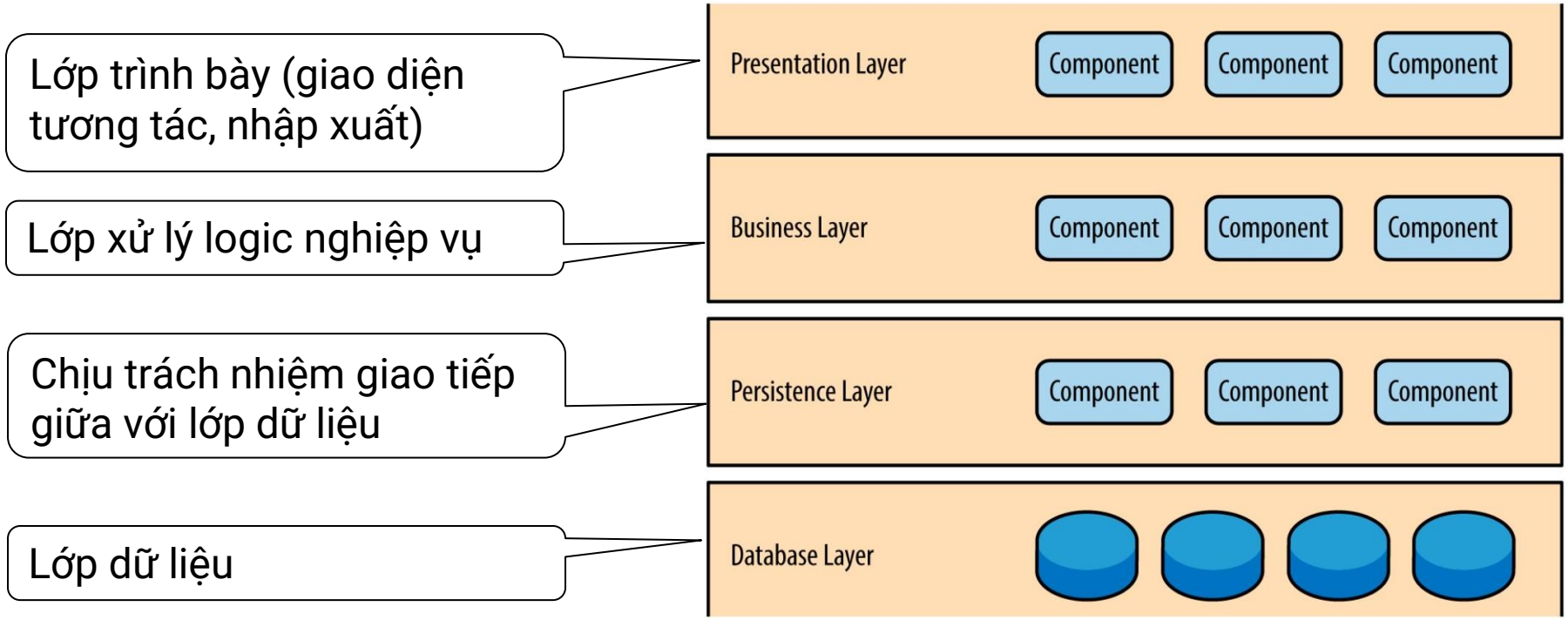
Vai trò của kiến trúc phần mềm

- Giảm thiểu rủi ro:
  - Kiến trúc thiết kế tốt giúp nhận diện và giảm thiểu rủi ro từ sớm trong quá trình phát triển, bao gồm vấn đề về hiệu suất, bảo mật và sự phụ thuộc giữa các thành phần.
- Tăng cường bảo mật:
  - Xác định cách bảo mật được tích hợp vào hệ thống, từ việc xác định các điểm truy cập đến các lớp bảo mật.
- Hỗ trợ quyết định kỹ thuật:
  - Cung cấp khung tham chiếu để đưa ra các quyết định từ việc chọn công nghệ đến cách triển khai các tính năng.
- Tăng khả năng hợp tác:
  - Giúp các nhà phát triển và các bên liên quan khác làm việc cùng nhau một cách hiệu quả hơn.

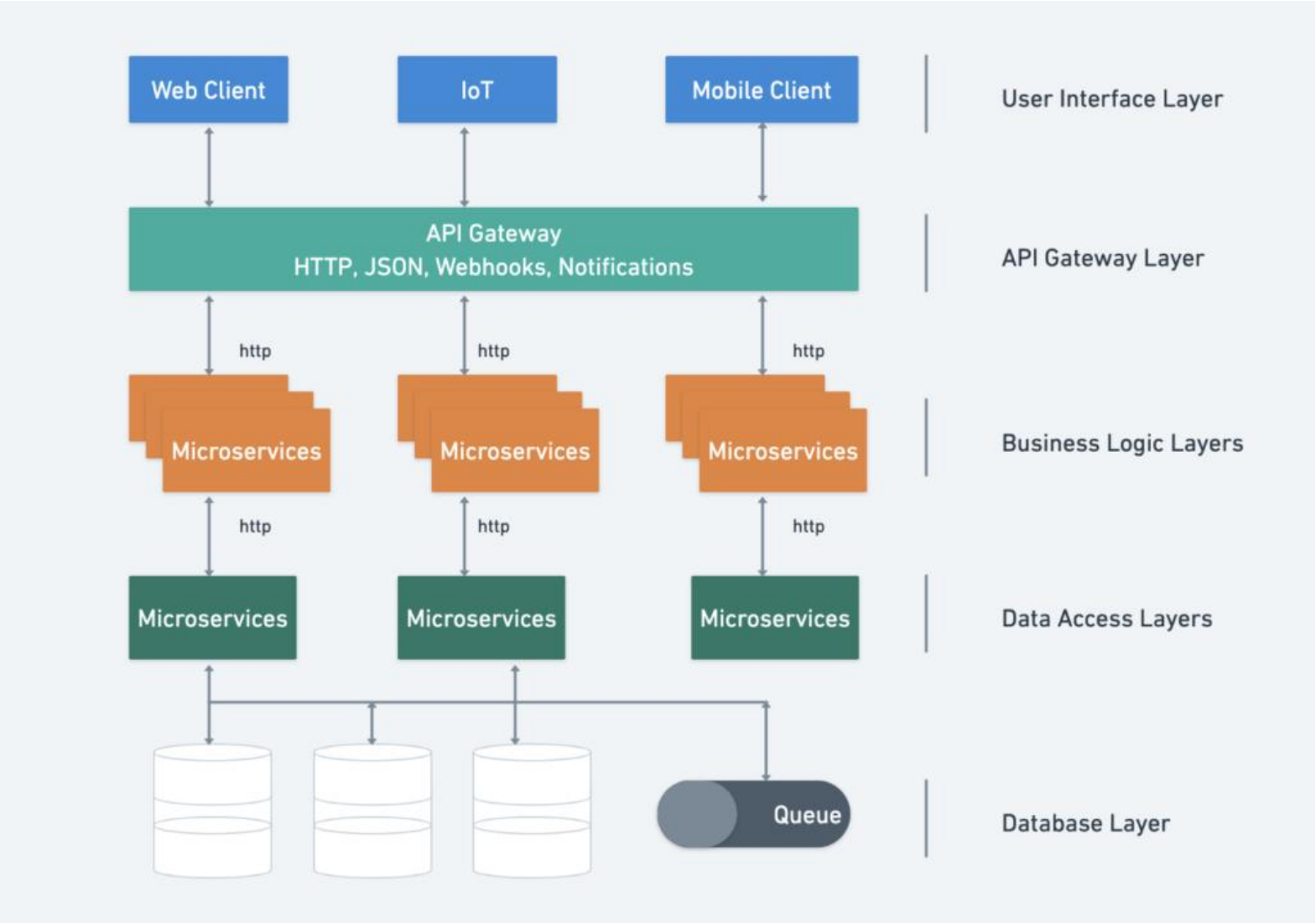

Architecture characteristics
Architect
Style
Component structure
Monolithic User Interface
API Layer
Service
Module
Database
Purchase
Order
Payment
Promotion
Common
Local
MakeOrder
Products
Deliver
Address
ManageInventory
Recipes
Location
Developer
Class design
User interface
Source code
LOGO
Heading

Kiến trúc phân lớp (Layered Architecture)

- Các thành phần trong phần mềm được tách thành các lớp có nhiệm vụ khác nhau và xếp chồng lên nhau.
- Các lớp độc lập với nhau, vì thế ta có thể sửa đổi mã bên trong một lớp mà không ảnh hưởng đến các lớp khác.



Sơ đồ kiến trúc phân lớp



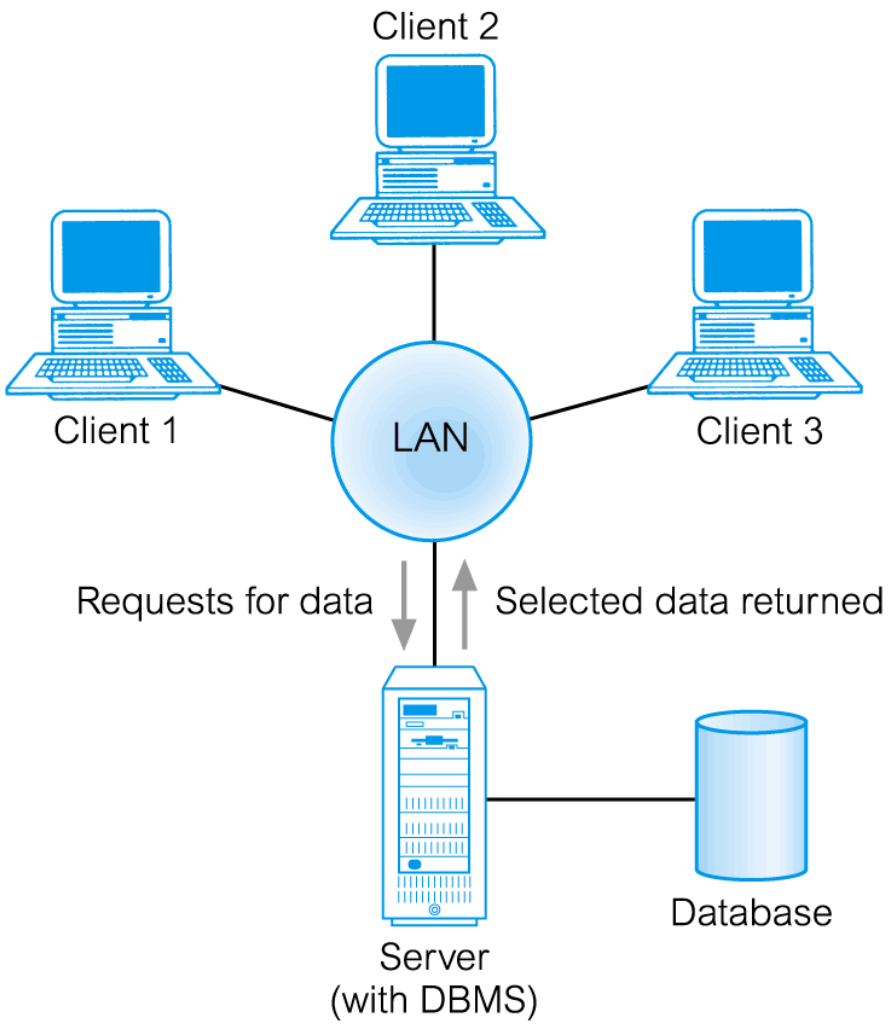
Phân biệt Layer và Tier

- Thường nói đến tier là nói đến lớp vật lý, còn layer là lớp logic (trừu tượng).
- Với ứng dụng đơn giản, chạy trên máy tính -> 1-tier
- Khi mô hình client / server phát triển -> tách thành 2-tier gồm app client (chứa UI, logic) kết nối đến server database. Các app như winform thuộc dạng này.
- Khi web phát triển -> tách thành 3-tier (client, server, database). Lúc này client (web browser) chỉ là view, xử lý logic đơn giản, còn server sẽ lo business logic.
- Mô hình 3-tier hiện vẫn phổ biến nhất. Dù app có phức tạp thêm thì người ta cũng ít chia thêm tier, thay vào đó thì tách source code ra thành các layer (layered architecture) và sử dụng thêm các pattern khác (MVC, MVVM, ...).

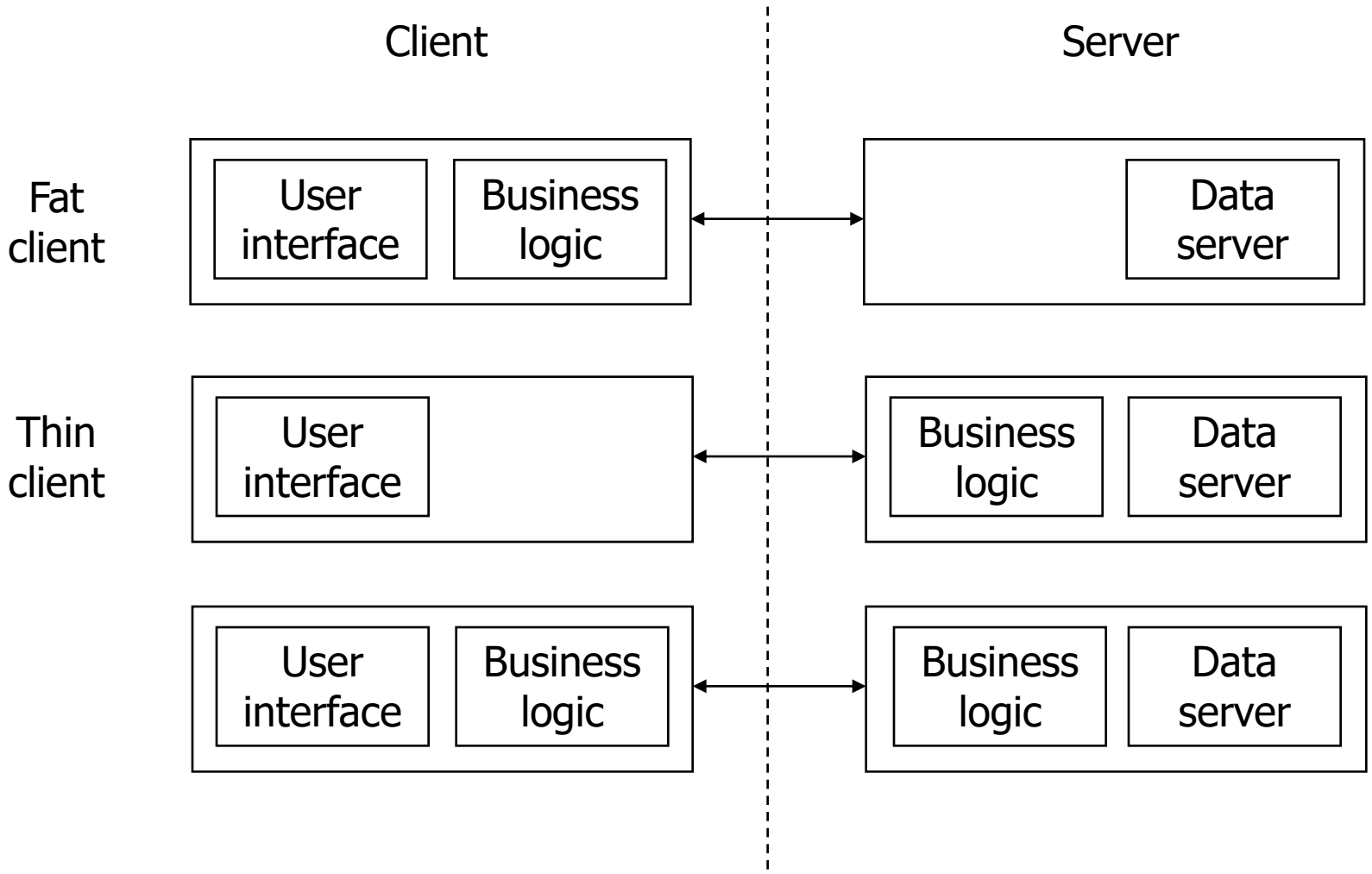
Phân biệt Layer và Tier

- Khi dùng từ layer, chúng ta nói tới việc phân chia ứng dụng thành các thành phần một cách logic theo chức năng hoặc theo vai trò.
- Các layer khác nhau khi được thực thi có thể nằm trong cùng một vùng bộ nhớ của một process, việc giao tiếp giữa 2 layer có thể không phải là giao tiếp giữa 2 process.
- Trái lại, tier liên quan đến cách phân chia một cách vật lý các thành phần trên các máy tính khác nhau.
- Điều làm ta nhầm lẫn giữa layer và tier là do chúng có cùng cách phân chia (presentation, business, data), tuy nhiên trên thực tế chúng khác nhau. Vì cách phân chia như trên nên 1 tier có thể chứa nhiều hơn 1 layer.

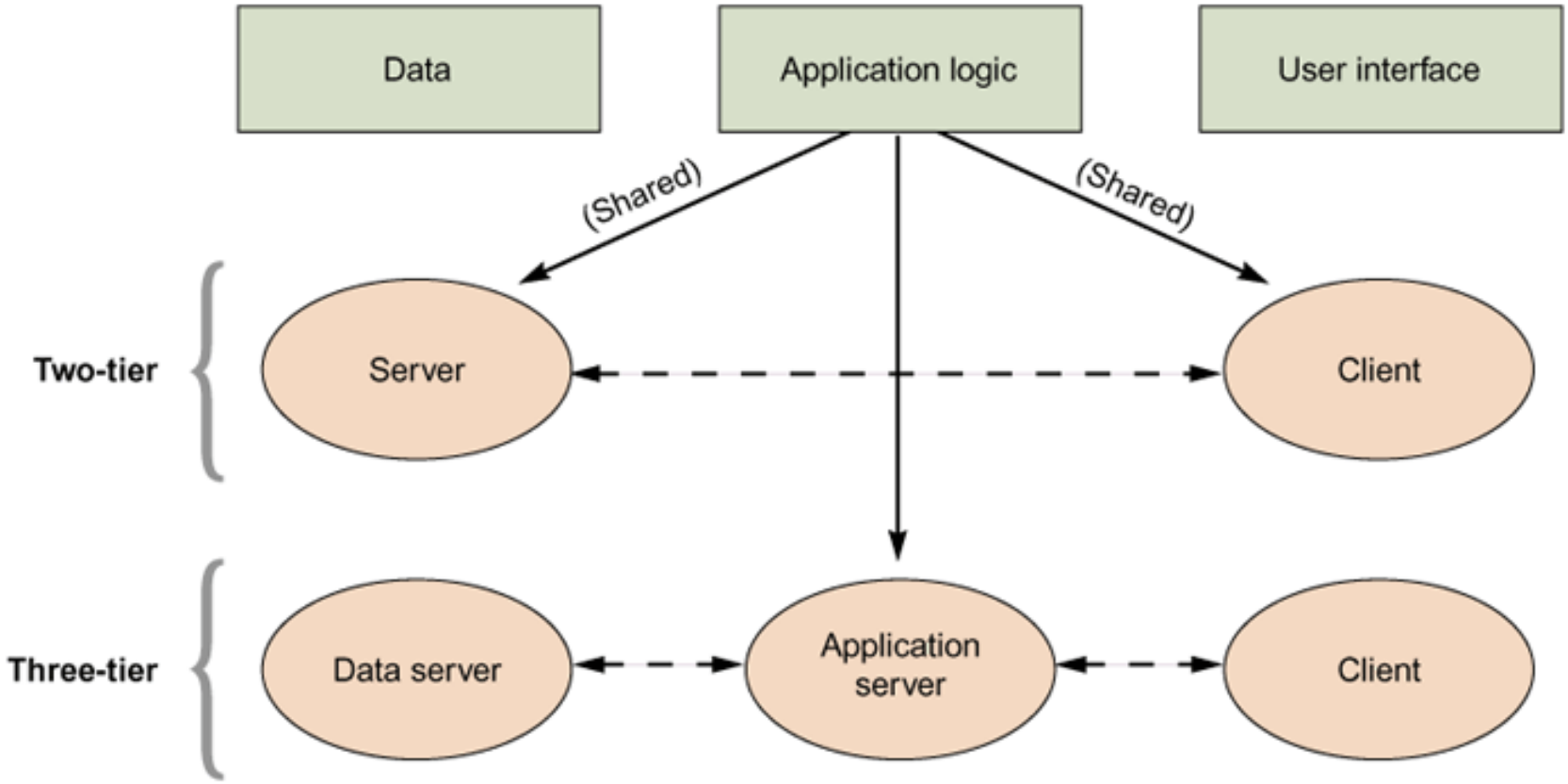
Kiến trúc 2-tier Client-Server



Kiến trúc 2-tier Client-Server



Kiến trúc 3-tier Client-Server



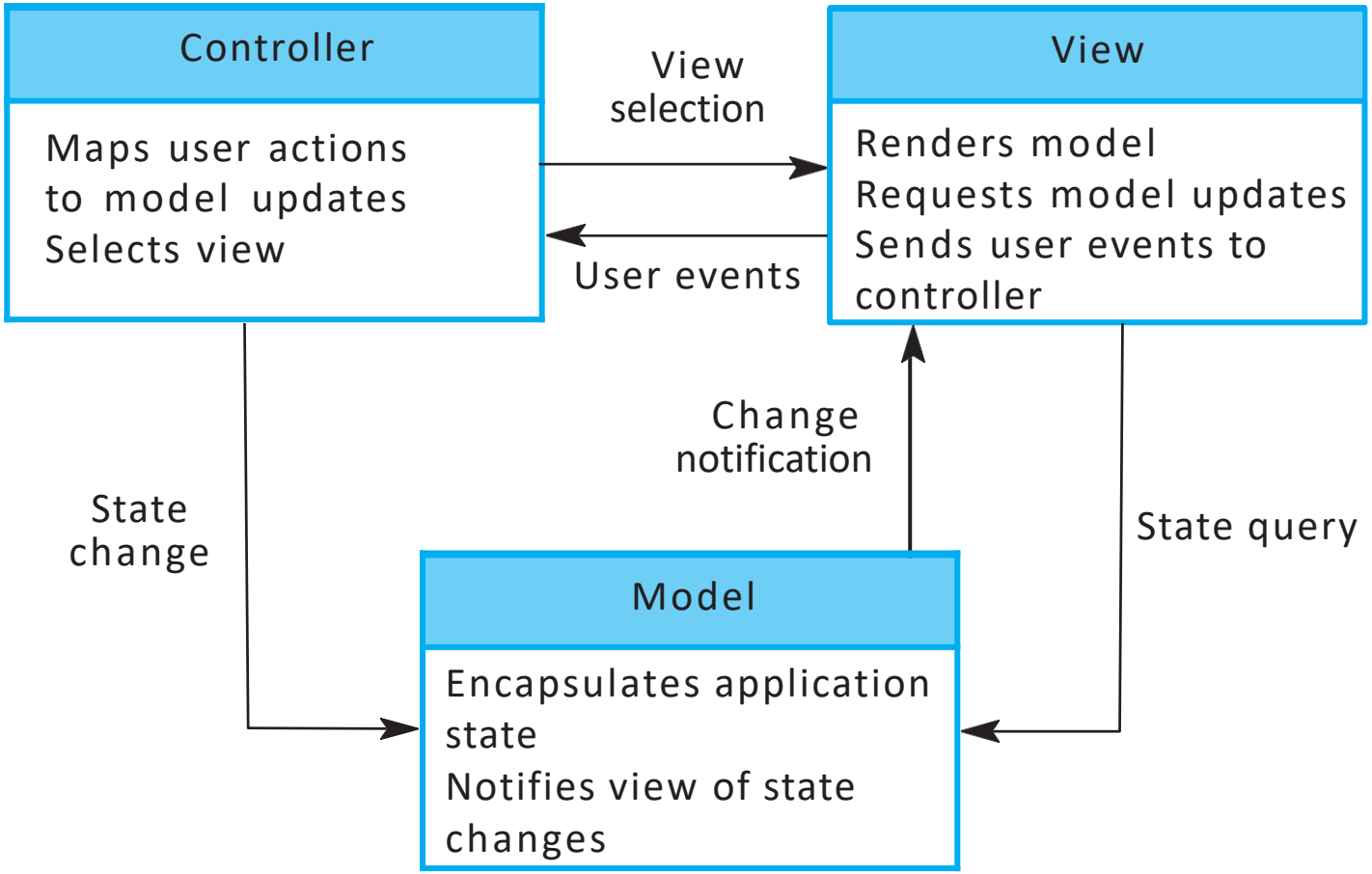
Kiến trúc MVC

- MVC (Model–View–Controller) là một mẫu kiến trúc phần mềm giúp tách biệt dữ liệu, giao diện và điều khiển, nhằm làm cho ứng dụng dễ bảo trì, mở rộng và kiểm thử hơn.
  - Model: Đại diện cho dữ liệu và logic nghiệp vụ. Nó quản lý trạng thái của ứng dụng, truy xuất và xử lý dữ liệu từ cơ sở dữ liệu hoặc API.
  - View: Thành phần hiển thị dữ liệu cho người dùng. View chỉ tập trung vào giao diện, không chứa logic nghiệp vụ.
  - Controller: Nhận yêu cầu từ người dùng qua View, xử lý bằng cách gọi Model, rồi trả kết quả về View.

Kiến trúc MVC

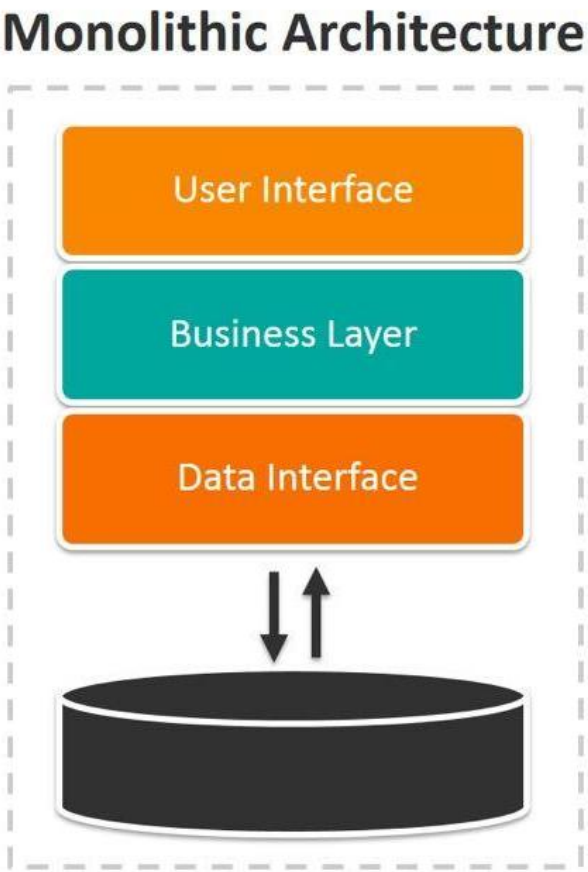
- Kiến trúc này được giới thiệu lần đầu vào năm 1979 và phổ biến trong các ứng dụng web, di động.
- Có nhiều framework triển khai theo mô hình này như:
  - Web frameworks: ASP.NET MVC, Spring MVC (Java), Ruby on Rails, Django (Python).
  - Mobile apps: trên iOS, Android
  - Desktop App: Smalltalk, Java Swing.
- MVC hiện vẫn là nền tảng cho nhiều kiến trúc như MVVM (Model–View–ViewModel) hay MVP (Model–View–Presenter), …

Mô hình MVC


Controller
Maps user actions to model updates
Selects view
View selection
User events
View
Renders model
Requests model updates
Sends user events to controller
Change notification
State change
State query
Model
Encapsulates application state
Notifies view of state changes

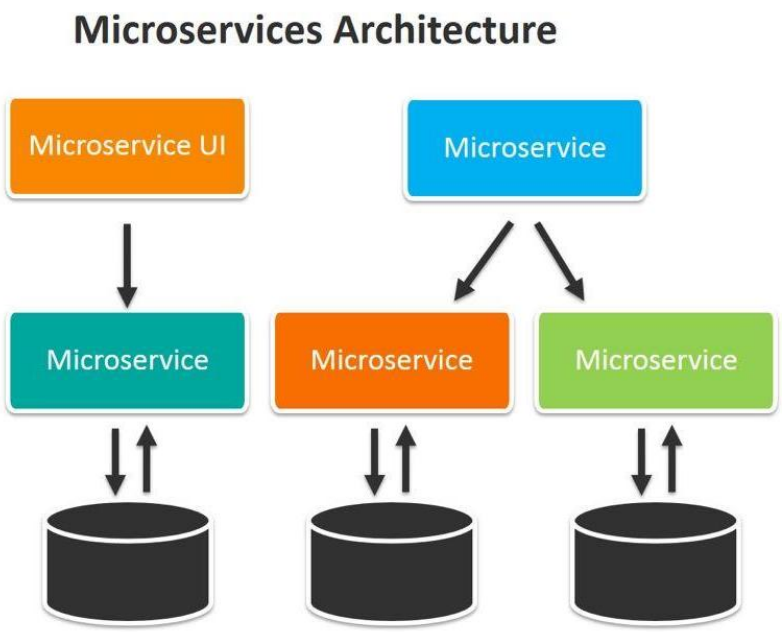
Kiến trúc nguyên khối (Monolith)

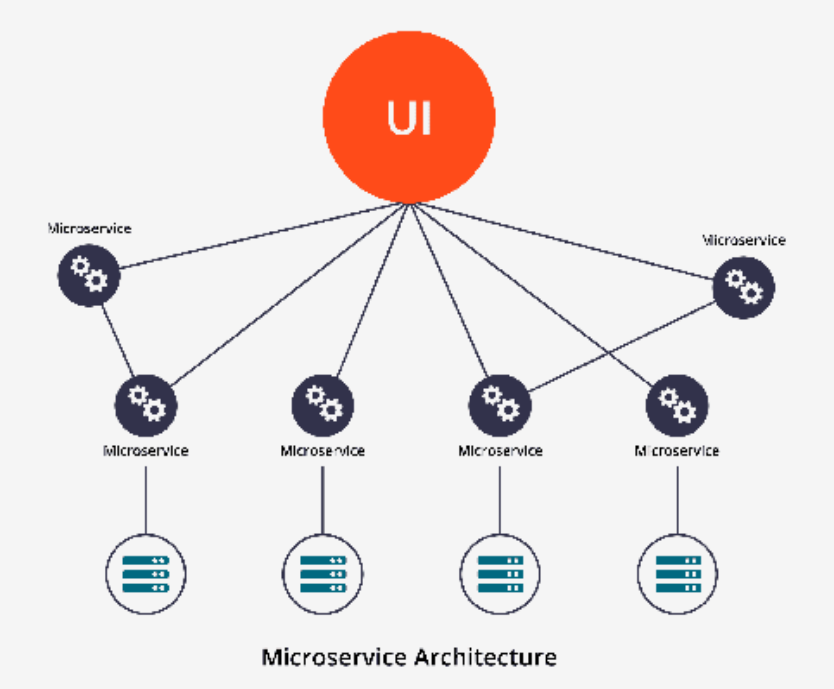
- Ưu điểm:
  - Tiết kiệm thời gian phát triển và triển khai 1 cách đơn giản;
  - Hiệu suất cao hơn do các thành phần giao tiếp với nhau 1 cách trực tiếp, đồng thời bộ nhớ được chia sẻ.
- Nhược điểm:
  - Khi lượng code, chức năng lớn dần thì hệ thống trở lên cồng kềnh và kém linh hoạt;
  - Nếu 1 thành phần trong hệ thống bị lỗi sẽ kéo theo việc toàn bộ hệ thống không thể hoạt động được.



Kiến trúc Microservices

- Các dịch vụ (chức năng) được triển khai riêng biệt, mỗi dịch vụ sẽ có một nhiệm vụ riêng. Các dịch vụ này độc lập với nhau để khi một dịch vụ lỗi thì các dịch vụ khác vẫn hoạt động bình thường.





Kiến trúc Microservice

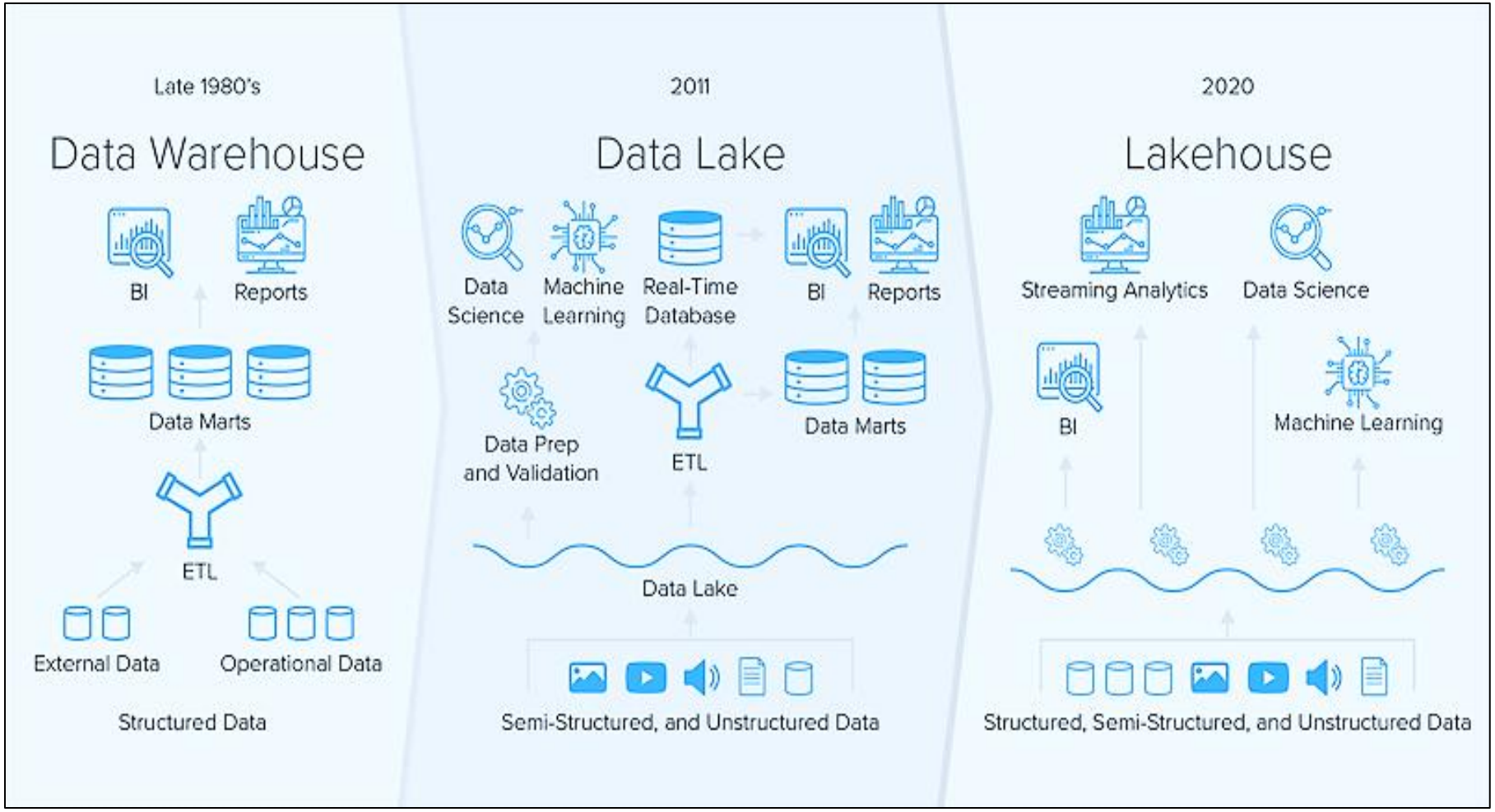
- Ưu điểm:
  - Dễ dàng triển khai, thử nghiệm các ứng dụng nhỏ 1 cách độc lập mà không làm ảnh hưởng đến hệ thống hiện có. Các service khác nhau cũng không bị tình trạng phải chờ đợi nhau hoàn thành mới có thể triển khai và chạy được, nhờ đó rủi ro khi triển khai cũng được giảm xuống trên đơn vị 1 service con.
  - Hệ thống có khả năng mở rộng theo chiều ngang, tức là dễ dàng nâng cấp, bổ sung phần cứng cho từng service với chi phí vừa phải, không cần đầu tư vào 1 máy có cấu hình cao như kiến trúc monolith để chạy hệ thống.

Kiến trúc Microservice

- Nhược điểm:
  - Phức tạp khi chia ứng dụng thành các service nhỏ độc lập khiến cần nhiều công việc quản lý hơn cho từng service đó.
  - Tính bảo mật cũng là 1 vấn đề khi các API sẽ phải public ra bên ngoài để các service khác có thể gọi tới, điều đó khiến nguy cơ bị tấn công cũng cao hơn.

Mô hình kiến trúc dữ liệu

Sự thay đổi của các mô hình kiến trúc dữ liệu



Data Warehouse là gì?

- Data warehousing là một CSDL được thiết kế nhằm vào mục tiêu truy vấn và phân tích hơn là xử lý các giao dịch.
- Thường chứa các dữ liệu lịch sử được lấy từ các dữ liệu giao dịch (transaction data), và có thể bao gồm dữ liệu từ nhiều nguồn khác nhau
- Môi trường datawarehouse bao gồm cả giải pháp ETL (extraction, transportation, transformation, and loading), OLAP (online analytical processing), data mining, các công cụ phân tích phía client, và các ứng dụng khác quản lý việc thu thập và chuyển dữ liệu đến người dùng

Các đặc điểm của data warehouse

- Subject Oriented

  Data warehouses được thiết kế để giúp phân tích dữ liệu → được xây dựng xoay quanh một chủ đề nào đó. Ví dụ: "Sales" giúp trả lời các câu hỏi như "các khách hàng tốt nhất trong năm là ai?", …

- Integrated

  Data warehouses sẽ lấy dữ liệu từ nhiều nguồn khác nhau về và tích hợp thành một định dạng xác định nào đó → phải giải quyết những vấn đề như xung đột về tên, sự không nhất quán giữa các đơn vị đo lường, …

Các đặc điểm của data warehouse

- Nonvolatile

  Một khi dữ liệu đã được đưa vào data warehouse thì nó không thể thay đổi → vì mục đích của data warehouse là giúp ta phân tích những gì đã xảy ra

- Time Variant

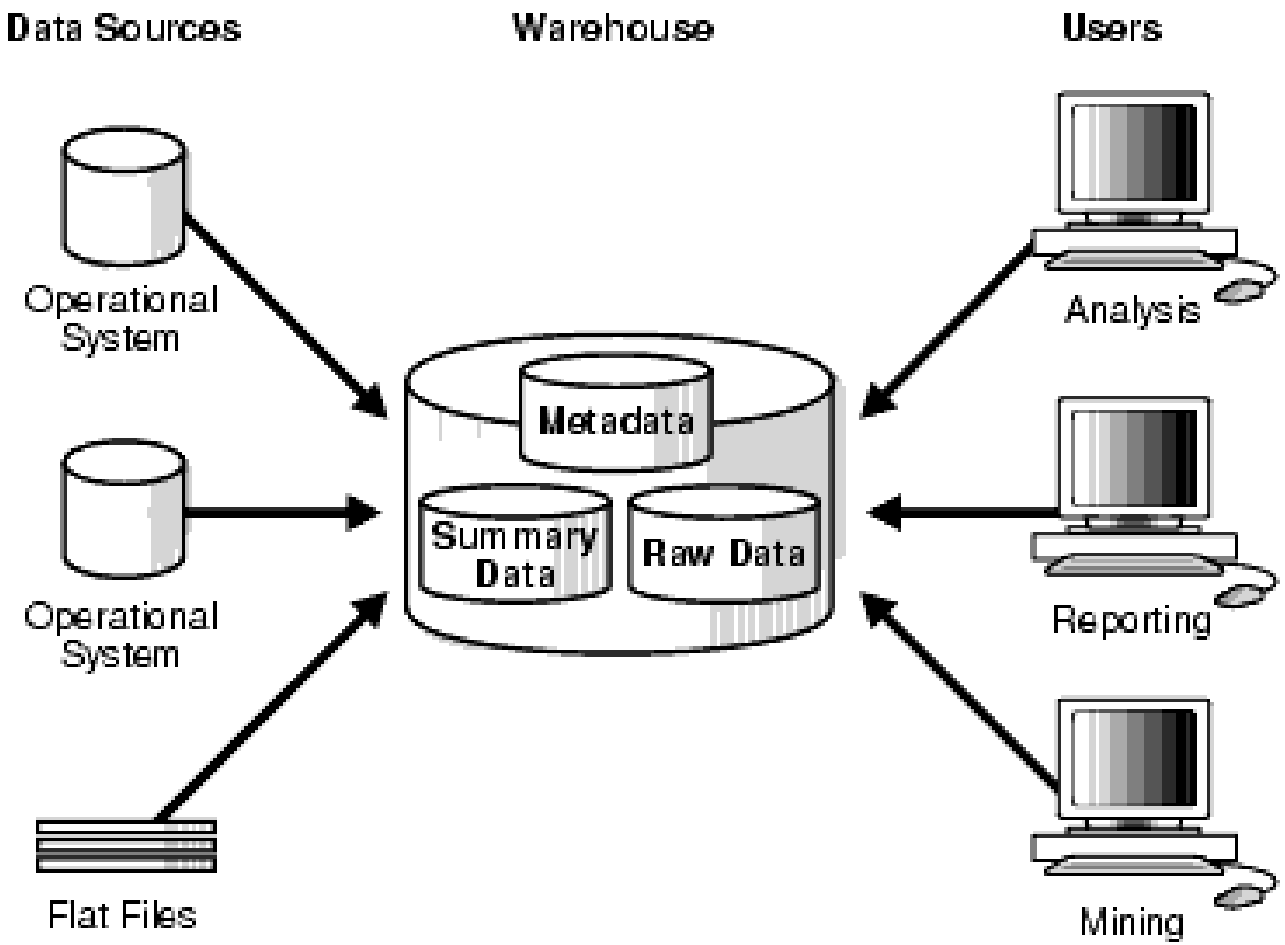
  Để phát hiện các xu hướng trong hoạt động nghiệp vụ cần một lượng dữ liệu lớn → quan tâm đến sự biến động của dữ liệu theo thời gian. Điều này khác với hệ thống OLTP (online transaction processing) do yêu cầu về hiệu suất nên dữ liệu lịch sử thường chuyển sang các kho lưu trữ khác

So sánh giữa data warehouse và OLTP

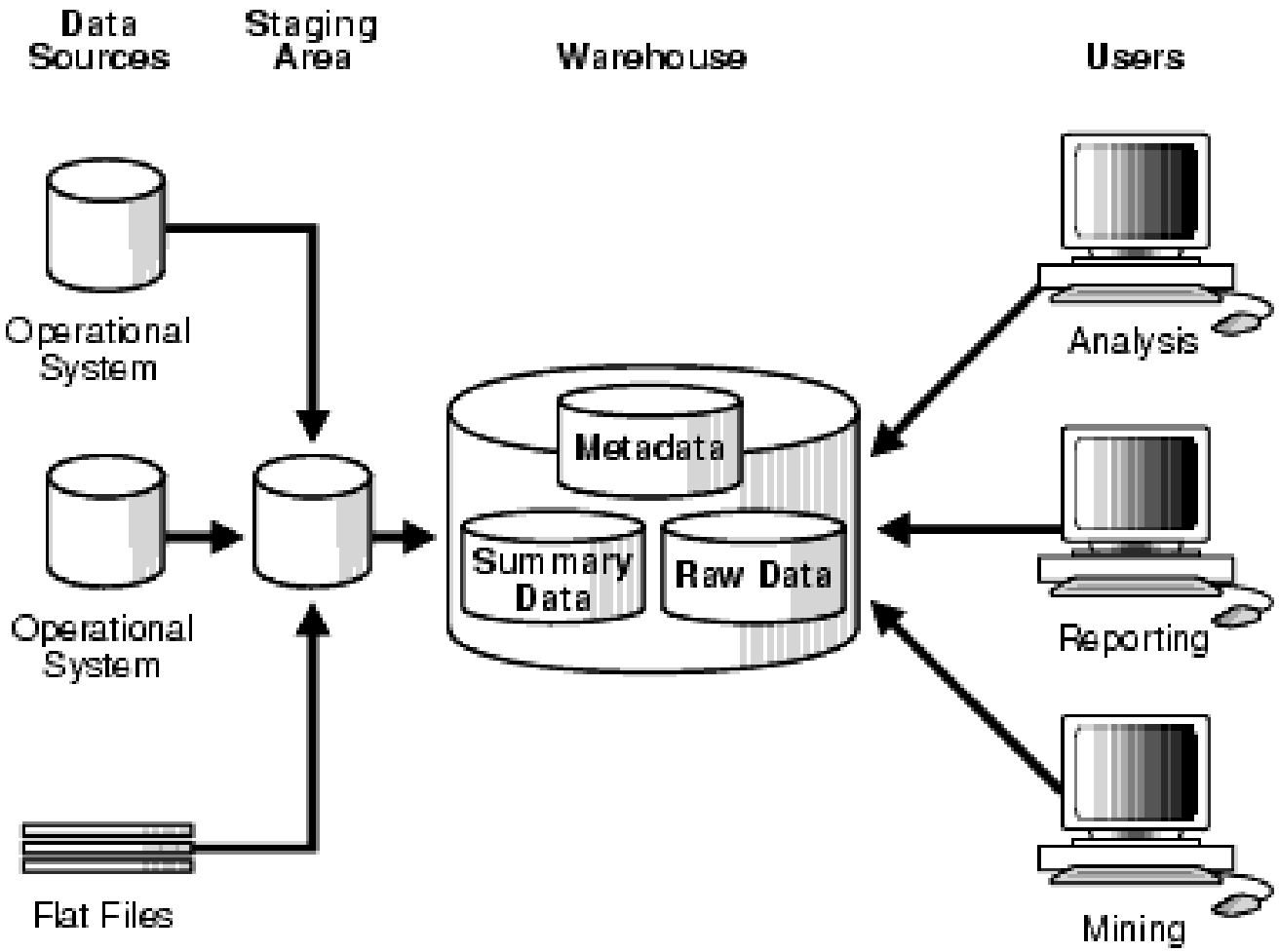
OLTP		Data Warehouse
Complex data structures (3NF databases)		Multidimensional data structures
Few	Indexes	Many
Many	Joins	Some
Normalized DBMS	Duplicated Data	Denormalized DBMS
Rare	Derived Data and Aggregates	Common

Một điểm cần lưu ý là dữ liệu trong data warehoudse thường không thuộc dạng chuẩn 3 (3NF), là dạng chuẩn thường gặp trong OLTP

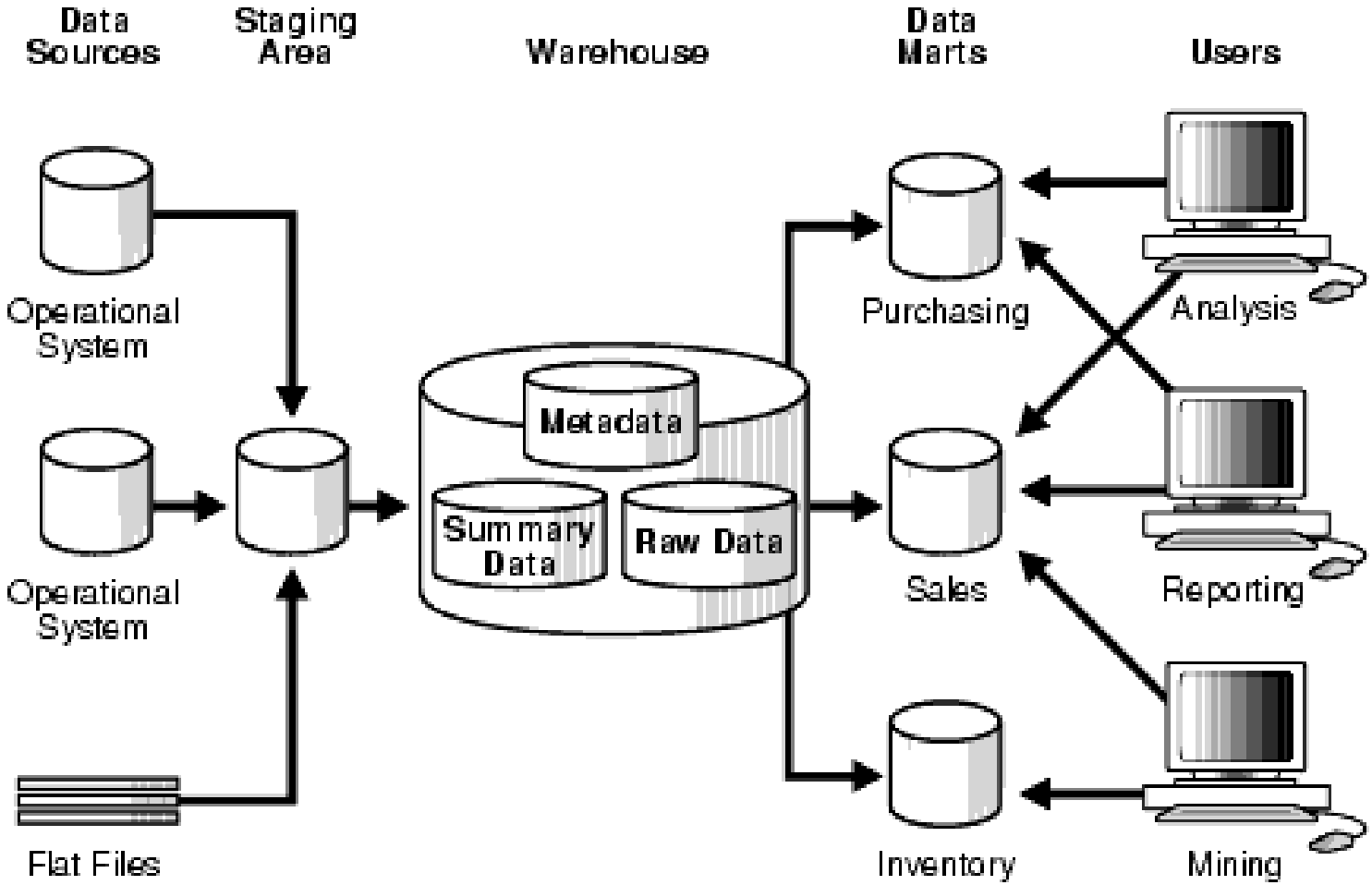
Data Warehouse Architecture (Basic)



Data Warehouse Architecture (with a Staging Area)

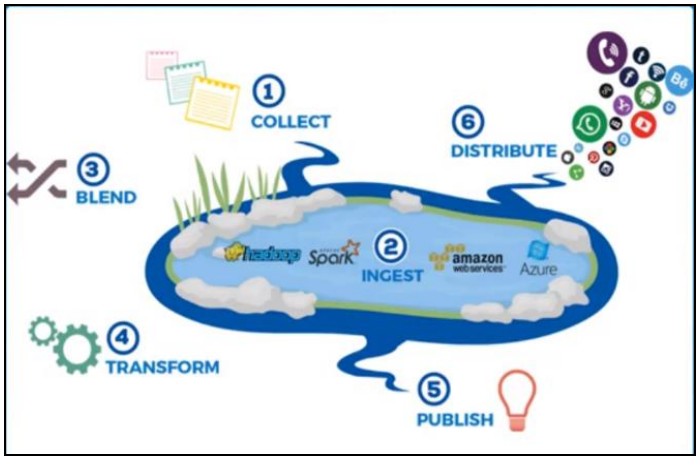


Data Warehouse Architecture (with a Staging Area and Data Marts)

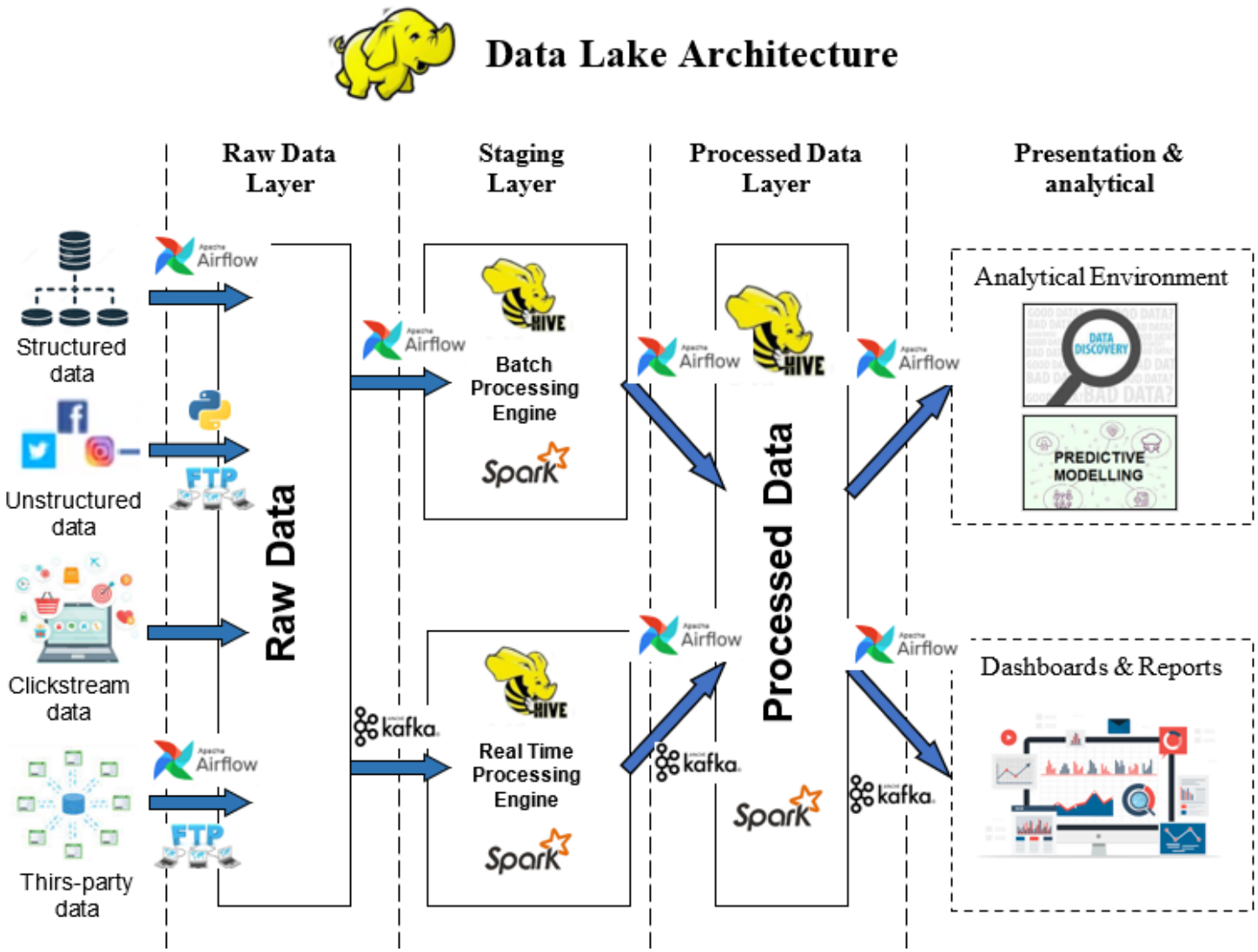


Data Lake

- Data Lake là nơi lưu trữ một lượng lớn các dữ liệu có cấu trúc (structured), bán cấu cấu (semi-structured) và phi cấu trúc (unstructured).
- Kiến trúc của data lake nhằm cung cấp dữ liệu cho nhiều loại người dùng khác nhau từ cấp quản lý, người dùng cuối, các nhà khoa học dữ liệu, các phân tích viên, ... do đó chất lượng của dữ liệu trong hồ rất quan trọng.



Kiến trúc Data Lake



Data Lake

- Về mặt dữ liệu, Data lake giữ dữ liệu thô ở trạng thái nguyên bản (raw data) chưa qua phân tích.
- Trong Data lake, tất cả các loại dữ liệu từ các nguồn của hệ thống đều được lưu trữ, bao gồm cả các nguồn dữ liệu như nhật ký web server, dữ liệu cảm biến, hoạt động trên mạng xã hội, văn bản, hình ảnh, ...
- Data lake có thể lưu trữ cả các dữ liệu hiện không sử dụng nhưng lại có thể cần thiết trong tương lai.
- Dữ liệu được lưu trữ ở dạng thô và luôn dễ dàng truy cập, cho phép tái cấu trúc mà không có trở ngại nào.
- Data lake giúp tận dụng tính linh hoạt của dữ liệu.

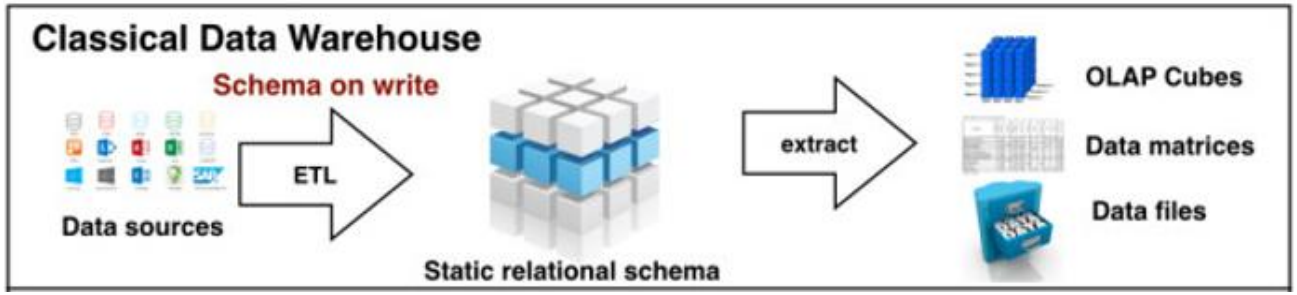
Data Lake

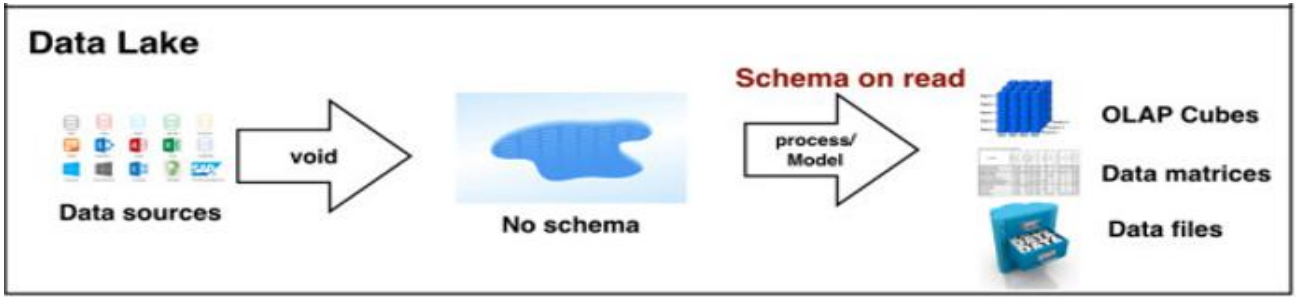
- Khi có nhu cầu sử dụng dữ liệu để giải quyết các bài toán nghiệp vụ, chỉ có những dữ liệu liên quan được chọn lọc và phân tích để đưa ra đáp án. Cách tiếp cận này gọi là "Schema on Read", giúp tiết kiệm thời gian và chi phí cho tổ chức.
- Data lake phù hợp với người dùng thực hiện phân tích chuyên sâu như các nhà khoa học dữ liệu. Với nhiều loại dữ liệu đa dạng trong data lake, người dùng có khả năng kết hợp nhiều loại dữ liệu khác nhau và đưa ra các câu hỏi hoàn toàn mới cần được giải đáp.
- Hệ sinh thái Hadoop rất phù hợp với kiến trúc lưu trữ này, và cũng giúp giảm chi phí lưu trữ.

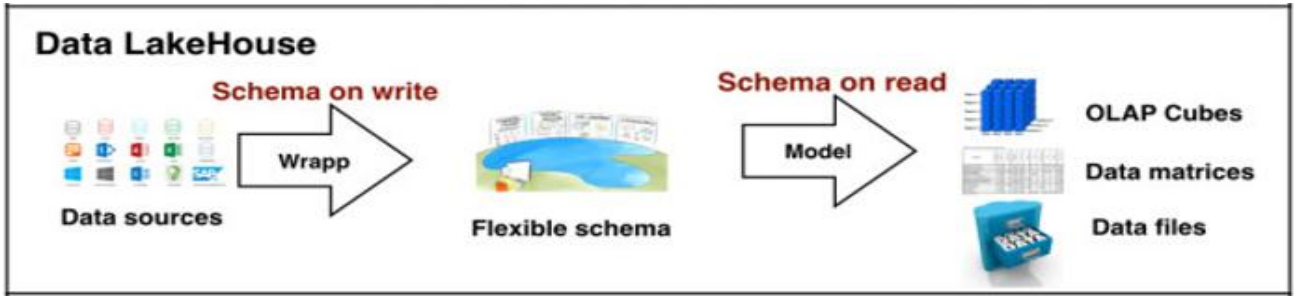
Data LakeHouse

- Kiến trúc Data LakeHouse đang được chấp nhận rộng rãi dù chỉ mới xuất hiện gần đây.
- Nhiều nghiên cứu cho thấy đây là kiến trúc tốt cho các hệ thống cơ sở dữ liệu lớn, sẽ được sử dụng nhiều trong thời gian sắp tới và ổn định trong một thời gian dài.

So sánh các mô hình kiến trúc dữ liệu

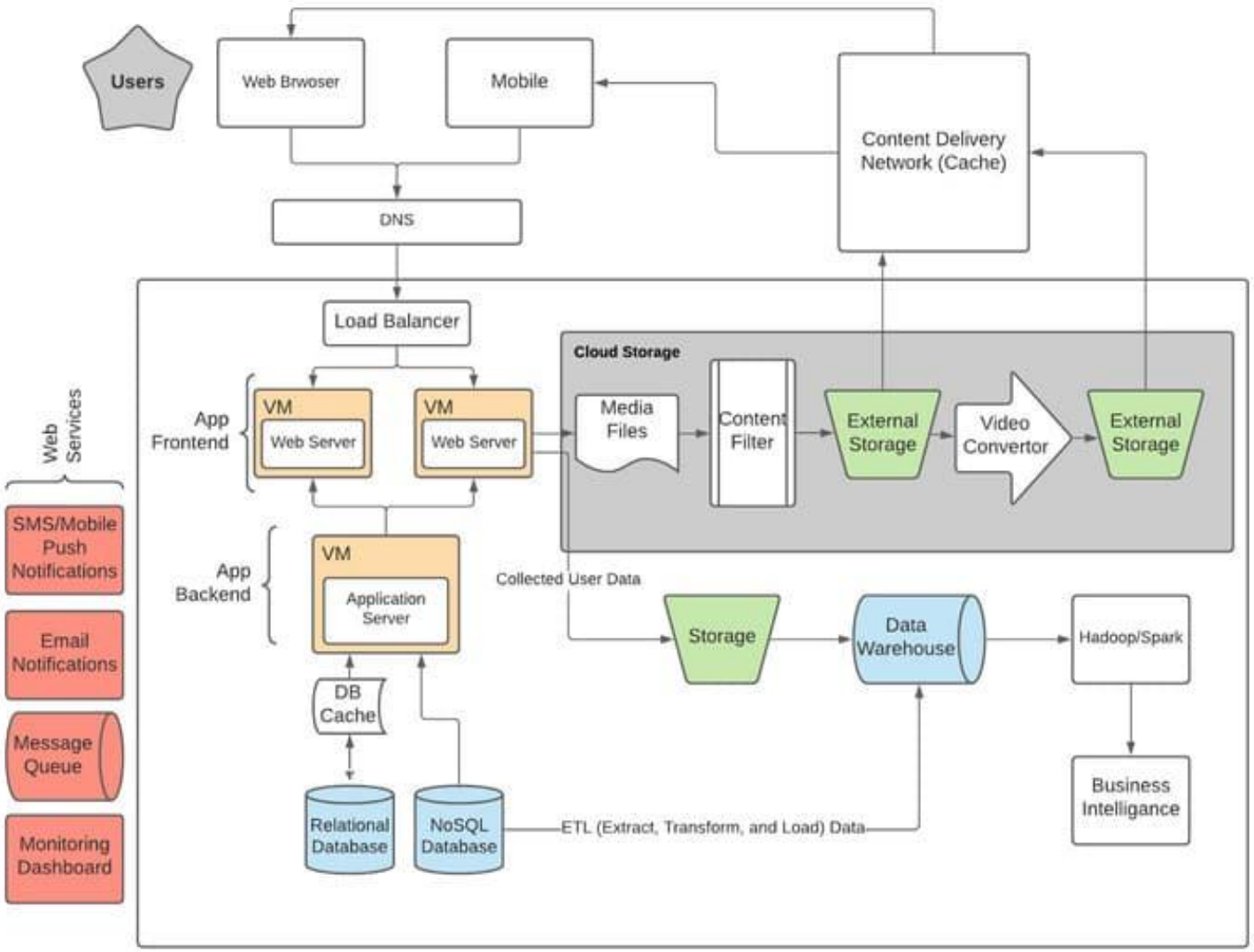






Data Mesh

- Data Mesh (lưới dữ liệu) nhằm mục đích giải quyết nhiều vấn đề tương tự như kết cấu dữ liệu (Data Fabric) - cụ thể là: khó khăn trong việc quản lý dữ liệu trong môi trường dữ liệu không đồng nhất - nó giải quyết vấn đề theo một cách cơ bản khác.
- Nói tóm lại, trong khi kết cấu dữ liệu (data fabric) tìm cách xây dựng một lớp quản lý ảo duy nhất trên dữ liệu phân tán, thì lưới dữ liệu (data mesh) khuyến khích các nhóm phân tán quản lý dữ liệu khi họ thấy phù hợp, mặc dù với một số quy định quản trị chung.
- Next Data Platform là Data Mesh? Data Mesh là "ngôi sao mới nổi" khi nhắc về kiểu lưu trữ dữ liệu hiện nay.

Marketing
Operational Applications
Analytical Applications
Data Products
Finance
Operational Applications
Analytical Applications
Data Products
Data Mesh
Common Data Product Modeling, Integration and Cataloging
Data Privacy
Data Governance
Service
Operational Applications
Analytical Applications
Data Products
Sales
Operational Applications
Analytical Applications
Data Products
Batch Data
Data Virtualization
Stream Data
Log-Based Data
Message-Based Data
API
RDBMS
Enterprise Apps
IoT
Data Hub
NoSQL Stores
SaaS Apps
Social Media
Data Lakes/ DWH

Sơ đồ kiến trúc ứng dụng web